

Orbit Core — Scaling Plan

Rodrigo Menchio + Claude

2026-03-26

Orbit Core — Performance Scaling Plan

Current Architecture (post v1.8.4)

Component	Config
PostgreSQL 16	Single instance, 1GB container, 512MB shm, shared_buffers 384MB
Connection Pool	Split: API pool (max 35) + Worker pool (max 15)
Workers	In-process (setInterval), 5 background workers
Cache	In-memory (5min TTL) + materialized catalog_event_cache
VPS	87–90% CPU steal time (oversold provider)
Partitioning	orbit_events, metric_points, metric_rollup_5m, metric_rollup_1h (all monthly)

Diagnosis: Bottlenecks

Bottleneck	Evidence
PG single-instance	No read replica, all reads + writes on same node
Workers in-process	A heavy rollup blocks the Express event loop
VPS with 87–90% CPU steal	No software optimization can fix oversold hardware
Zero distributed cache	In-memory cache dies on restart, no sharing across instances
Correlate worker	3 CTEs with STDDEV + JOIN every 5min over raw data

Phase 1 — Quick Wins (v1.8.4 — DONE)

1.1 Separate Pool: API vs Workers — DONE

Two `pg.Pool` instances:

- **poolApi** (max 35, 30s timeout): Serves all HTTP route handlers
- **poolWorker** (max 15, 120s timeout): Serves rollup, correlate, alerts, threat-intel, connectors

Impact: Workers can never starve API queries. Pool saturation warnings are actionable per-pool.

1.2 Partition Rollup Tables — DONE

`metric_rollup_5m` and `metric_rollup_1h` converted to range-partitioned by `bucket_ts` (monthly, 2025–2027).

Impact: INSERT ON CONFLICT scans only the current month's partition instead of the full 90/180-day table. Retention purge becomes O(1) DROP PARTITION.

1.3 Materialize Catalog Queries — DONE

`catalog_event_cache` table refreshed by the rollup worker every 5 min. The `/catalog/events` endpoint reads from cache instead of 4 parallel GROUP BY queries.

Impact: Catalog response from ~500ms to <5ms.

1.4 Increase PG Container Resources — DONE

- Container memory: 512MB to 1GB
- `shm_size`: 256MB to 512MB
- `shared_buffers`: 384MB, `effective_cache_size`: 768MB, `work_mem`: 8MB

Impact: More data stays in PG buffer cache, fewer disk reads.

Phase 2 — Separation of Concerns (pending)

2.1 Workers as Separate Process

Extract rollup, correlate, and threat-intel workers into an independent `orbit-worker` Docker service. Own memory limit, own pool, no event loop contention with Express.

Effort: Medium — refactor worker startup into a standalone entry point, add Docker service, share DB config.

2.2 Redis Cache Layer

Add Redis for frequently-accessed data (catalog, wazuh summary, RAG catalog). Survives restarts, shareable across multiple API instances.

Effort: Medium — add Redis service to docker-compose, replace in-memory caches with Redis get/set + TTL.

2.3 Read Replica

PG streaming replication: API reads from replica, workers write to primary. Doubles read capacity without touching query code.

Effort: Medium — add replica PG container, configure streaming replication, route read-only queries to replica pool.

2.4 PgBouncer

Connection pooler in front of PG (transaction mode). Hundreds of virtual connections with ~30 real PG connections. Essential for horizontal scaling.

Effort: Low — add PgBouncer container, point API/Worker pools to PgBouncer instead of PG directly.

Phase 3 — Real Scale (pending)

3.1 TimescaleDB

Drop-in PG replacement. Hypertables with native compression (10–20x disk savings), continuous aggregates replace manual rollups, automatic retention policies. Migration is nearly transparent.

Effort: High — replace PG image with TimescaleDB, convert tables to hypertables, rewrite rollup logic to use continuous aggregates.

3.2 Dedicated VPS

With 87–90% steal time, hardware is the real bottleneck. A dedicated 4 vCPU + 8GB RAM VPS (Hetzner/OVH ~EUR 25/month) changes the game entirely.

Effort: Low — provision new VPS, migrate Docker stack, update DNS.

3.3 Queue-Based Ingest (BullMQ + Redis)

Ingest endpoint accepts, enqueues, returns 202. Worker processes in optimized batches. Decouples API latency from write throughput.

Effort: Medium — add BullMQ dependency, refactor ingest handlers to enqueue, add consumer worker.

3.4 Horizontal API Scaling

With Redis cache + PgBouncer + read replica, running 2–3 API instances behind Traefik is trivial. Stateless API, shared cache, pooled connections.

Effort: Low — add replicas in Docker Swarm, Traefik already handles load balancing.

Progress Tracker

Item	Impact	Effort	Status
1.1 Split pools	High	Low	DONE (v1.8.4)
1.2 Partition rollups	High	Low	DONE (v1.8.4)
1.3 Catalog cache	Medium	Low	DONE (v1.8.4)
1.4 PG resources	Medium	Trivial	DONE (v1.8.4)
2.1 Worker process	High	Medium	Pending
2.2 Redis	Medium	Medium	Pending
2.3 Read replica	High	Medium	Pending
2.4 PgBouncer	Medium	Low	Pending
3.1 TimescaleDB	Very High	High	Pending

Item	Impact	Effort	Status
3.2 Dedicated VPS	Very High	Low	Pending — highest ROI
3.3 Queue ingest	High	Medium	Pending
3.4 Horizontal API	High	Low	Pending (after 2.x)

Generated 2026-03-26 by Rodrigo Menchio + Claude Code — Updated after v1.8.4 release